Exam-Slot Assignment with QAOA and Simulated Annealing: Part II

Group G3: Alef Keuffer, Diogo Ferreira, Luis Arruda

November 11, 2026

1 Introduction

Our group was assigned the exam-slot assignment problem, which describes the process of scheduling *exams* within a given number of *batches* while avoiding conflicts, a conflict being a *student* having to attend several *exams* that take place in the same *batch*.

Inspiration. At the start, we struggled to understand the problem's context and how to formulate it. Afterwards, we found a very useful paper that deals with the same problem as ours, "Hybrid quantum annealing for large-scale exam scheduling" [7]. We based the output of our model-to-text transformations on it: the generated scripts of both chains solve a *QUBO* over the paper's *assignment variables* [7, §5.3, Eq. (4)], the BESSER chain's in the paper's formulation, the EMF chain's in a weighted variant of it (Section A). It helped us formulate the problem and its constraints. Our solutions were heavily inspired by the way the paper framed the problem: transforming students' course registrations into a course conflict matrix [7, §5.2], which the EMF chain models as a *Conflict Graph*.

Strategy. We decided to implement two chains, one using BESSER and one using EMF, to compare the two technologies and to get the whole team familiar with the problem and the implementation process. More details on the chains can be found in Sections 2 and 3.

The EMF chain's terms are those of the glossary (Section F); the BESSER chain's sections keep its own words: *slot* for a *Batch*, *valid* for a *schedule* that places every *exam* without a *Conflict*.

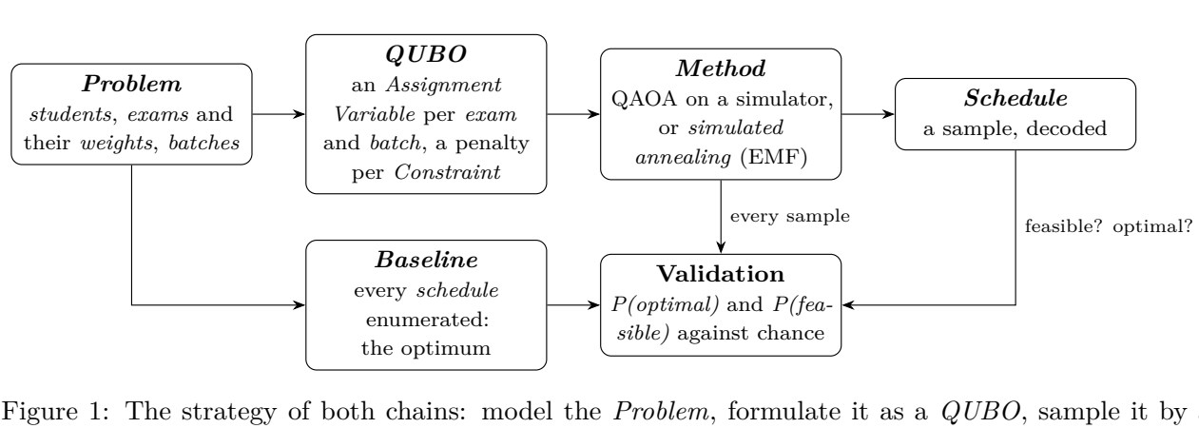
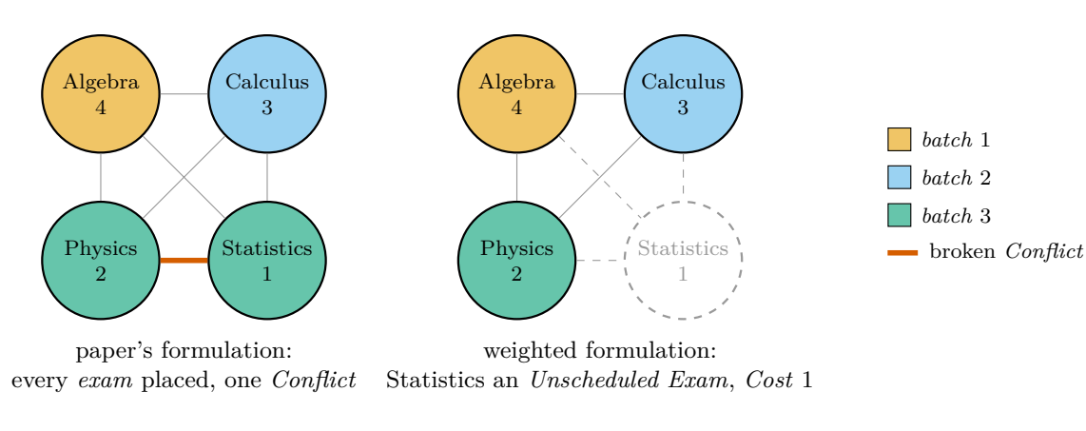

Problem
students, exams and their weights, batches
QUBO
an Assignment Variable per exam and batch, a penalty per Constraint
Method
QAOA on a simulator, or simulated annealing (EMF)
Schedule
a sample, decoded
every sample
feasible? optimal?
Baseline
every schedule enumerated: the optimum
Validation
P(optimal) and P(feasible) against chance

Figure 1: The strategy of both chains: model the *Problem*, formulate it as a *QUBO*, sample it by a *Method*, decode a sample into a *Schedule*, and judge every sample against a *Baseline* that enumerates every *schedule*.

1.1 Two chains

The strategy is implemented twice, independently: the EMF chain in `emf/`, with Xtext, ATL and Acceleo (Section 2), and the BESSER chain in `besser/`, with Python transformations over B-UML models and BESSER's Qiskit generator (Section 3). They solve the two formulations of Section A, so only the EMF chain's results depend on the *Weight*.



paper's formulation: every *exam* placed, one *Conflict*

weighted formulation: Statistics an *Unscheduled Exam*, *Cost* 1

Figure 2: The two formulations' optima on `clique`: four *exams*, with their *weights*, that all share every *student*, in three *batches*. A node's colour is its *batch*; an edge, a *Conflict*. The paper's formulation places every *exam* and breaks one *Conflict*, any two *exams* sharing a *batch* at the same energy; the weighted one leaves the lightest *exam*, Statistics, an *Unscheduled Exam*, at *Cost* 1.

2 EMF chain

One *problem*, followed end to end. Take the smallest instance, `emf/instances/1-path.exam`: four *exams* whose *conflicts* form a path, two *batches*.

1-path.exam

```
// Q16 (i): four exams whose conflicts form a path, two batches. Feasible (a path is 2-colourable).
problem path batches 2
students Ana Rui Eva
exam Algebra   weight 2 { Ana }
exam Calculus  weight 3 { Ana Rui }
exam Physics   weight 1 { Rui Eva }
exam Chemistry weight 2 { Eva }

solver { method QAOA depth 2 optimizer COBYLA iterations 150 shots 2048 seed 7 }
```

Running `cd emf && ./build.sh` turns that text into the files in `emf/out/path/`. What happens to it, in order:

1. `1-path.exam` (text).
2. Xtext: the parser from `Exam.xtext` (text → model) yields `problem.xmi` and `solverconfig.xmi`, which conform to `problem.ecore` and `solverconfig.ecore`.
3. OCL validation, then ATL `problem2conflictgraph.atl` (model → model) yields `conflictgraph.xmi`, which conforms to `conflictgraph.ecore`.
4. OCL validation, then Acceleo `main.mtl` (model → text) yields `solve_path.py`.
5. Python at run time: pyqubo builds the *QUBO*, then QAOA or annealing samples it, writing `schedule_path.xmi` and a printed report; the *schedule* conforms to `schedule.ecore`.

The Java that glues the first three arrows is one class, `ChainApplication.java` in the chain bundle (`emf/bundles/pt.up.feup.examschedule.chain/`). Its `run` method is the chain above: parse, validate, save, ATL, save, validate, Acceleo. The grammar, ATL module, Acceleo template and runtime are expanded in Section D.

Three metalevels. Each `.xmi` file is a *model*. It is only meaningful because it *conforms to* a *metamodel*, an `.ecore` file, which in turn is written in Ecore, EMF's metamodelling language; XMI is how EMF serialises those models.

Level	In this repo	Says
Meta-metamodel	Ecore itself	there are classes, attributes, references
Metamodel	`problem.ecore`	a *Problem* has *exams*; an *Exam* has *students*
Model	`emf/out/path/problem.xmi`	Calculus is sat by Ana and Rui

The OCL constraints (`uniqueExamNames`, `atLeastOneBatch`,...) live *inside* the metamodel as annotations, so they travel with it and every tool that loads `problem.ecore` can check them (Section E).

Three kinds of arrow. Model-driven engineering names transformations by what goes in and out. The chain has one of each:

- **Text → model**: Xtext. The grammar does not invent its own classes; its rules say `returns problem::Problem`, so the parser builds instances of the hand-written metamodel.
- **Model → model**: ATL. Rules map a *Problem* to a *Conflict Graph* and add an edge for each pair of *exams* sharing a *student*, the paper's Algorithm 1 [7, §5.2, Algorithm 1].
- **Model → text**: Acceleo. A template walks the *Conflict Graph* and prints Python.

The last model the chain itself produces is the *Conflict Graph*. The *QUBO* never becomes an `.xmi`: pyqubo builds it in memory when the generated script runs (`emf/docs/adr/0007`), the hand-off chosen in Section A.

3 BESSER chain

The BESSER chain takes a description of the *exams*, transforms it step by step into a quantum circuit, runs that circuit on a simulator, and converts the results back into a timetable that is easy for a person to read. Every step writes its result to `besser/output/<instance>/`, one file per step, so each step can be opened and checked independently. Figure 3 illustrates the smallest example, `tiny` (3 *exams*, 3 time slots, 9 qubits).

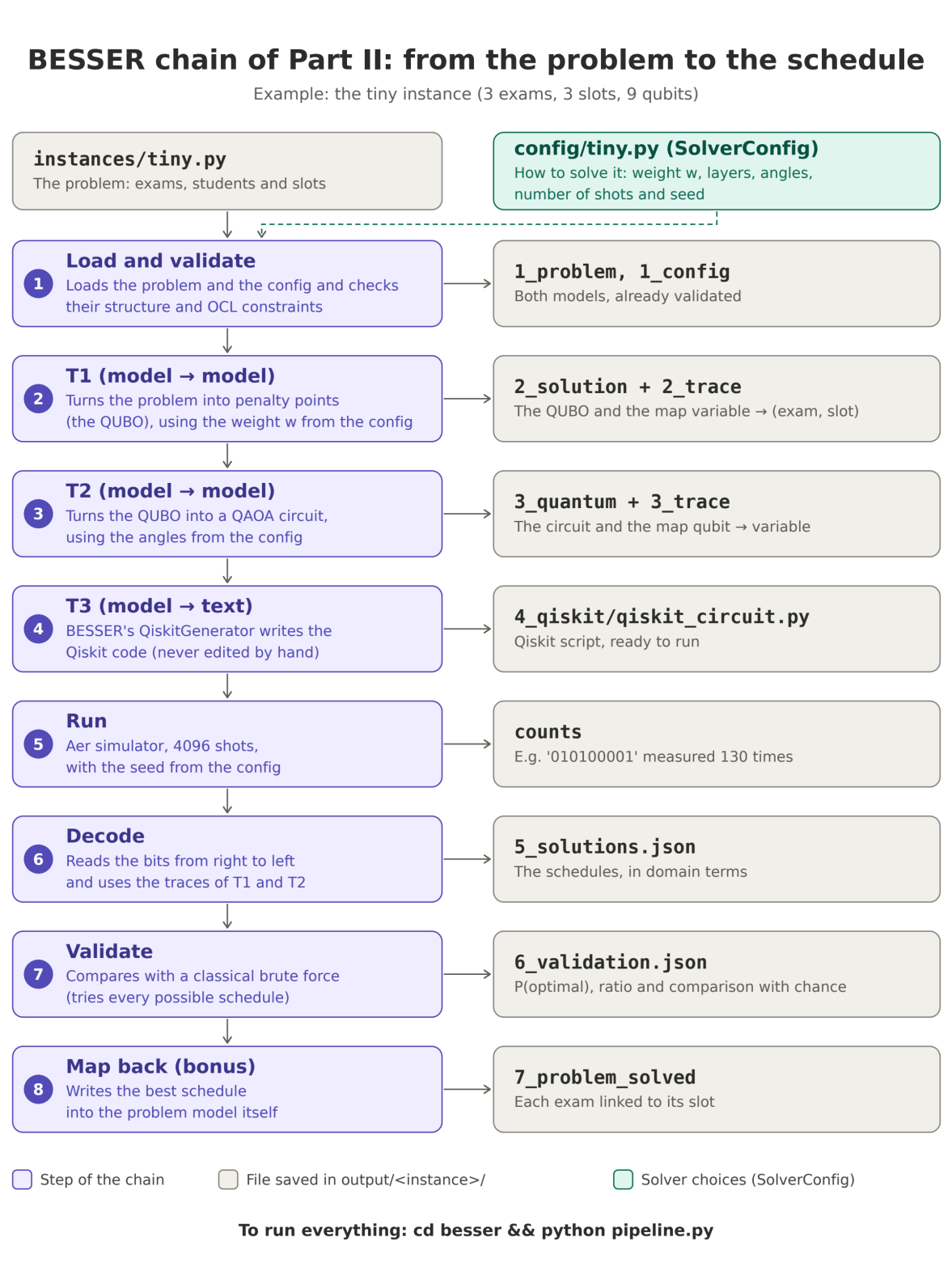

BESSER chain of Part II: from the problem to the schedule
Example: the tiny instance (3 exams, 3 slots, 9 qubits)
instances/tiny.py
The problem: exams, students and slots
config/tiny.py (SolverConfig)
How to solve it: weight w, layers, angles, number of shots and seed
1 Load and validate
Loads the problem and the config and checks their structure and OCL constraints
1_problem, 1_config
Both models, already validated
2 T1 (model → model)
Turns the problem into penalty points (the QUBO), using the weight w from the config
2_solution + 2_trace
The QUBO and the map variable → (exam, slot)
3 T2 (model → model)
Turns the QUBO into a QAOA circuit, using the angles from the config
3_quantum + 3_trace
The circuit and the map qubit → variable
4 T3 (model → text)
BESSER's QiskitGenerator writes the Qiskit code (never edited by hand)
4_qiskit/qiskit_circuit.py
Qiskit script, ready to run
5 Run
Aer simulator, 4096 shots, with the seed from the config
counts
E.g. '010100001' measured 130 times
6 Decode
Reads the bits from right to left and uses the traces of T1 and T2
5_solutions.json
The schedules, in domain terms
7 Validate
Compares with a classical brute force (tries every possible schedule)
6_validation.json
P(optimal), ratio and comparison with chance
8 Map back (bonus)
Writes the best schedule into the problem model itself
7_problem_solved
Each exam linked to its slot
Step of the chain
File saved in output/<instance>/
Solver choices (SolverConfig)
To run everything: cd besser && python pipeline.py

Figure 3: The BESSER chain on the `tiny` instance: each step (left) and the model it saves (right).

Two inputs: the *what* and the *how*. Everything starts with two files written by a person. The first one describes the *problem*: the *exams*, the *students* taking them, and the time slots (`besser/instances/tiny.py`, a model of Part I's Problem DSL). The second one defines *how to solve it*: the cost of a *conflict*, the number of layers in the circuit, its angles, how many times it should run, and a seed so that every execution produces the same result (`besser/config/tiny.py`, a model of our *Solver Configuration* metamodel). They are kept separate on purpose: to solve the same *problem* in a different way, only the configuration changes, never the code.

1. **Read and check.** Both models are loaded and validated against the rules of their metamodels, written in OCL: for instance, an *exam* must have a name, and the number of circuit layers must match the depth. If a rule fails, the chain stops here. Saved: `1_problem`, `1_config`.
2. **T1, model to model: from *exams* to penalty points.** The *problem* becomes a *QUBO*, a list of penalty points. There is one yes/no variable for each pair (*exam*, slot), and a *schedule* accumulates points for every rule it violates: an *exam* with no slot or with two slots, or two *exams* sharing a *student* placed in the same slot [7, §5.3]. A *schedule* with zero points is a valid one. Leaving an *exam* out always costs more than a *conflict*, so the solver cannot "cheat" by dropping an *exam* [4, §2.3]. Saved: `2_solution`, and a trace linking each variable to its *exam* and slot.
3. **T2, model to model: from penalty points to a quantum circuit.** The *QUBO* becomes a *QAOA circuit* [1] in BESSER's own quantum metamodel: one qubit per variable, and layers that steer the qubits towards the *schedules* with fewer points. The angles of each layer come from the configuration. Saved: `3_quantum`, and a trace linking each qubit to its variable.
4. **T3, model to text: writing the code.** BESSER's `QiskitGenerator` writes the circuit as a Qiskit script, `besser/output/<instance>/4_qiskit/qiskit_circuit.py`. The generated code is never edited by hand: if something is wrong, the model or the transformation is fixed instead.
5. **Run.** The script runs on the Qiskit Aer simulator [3] 4096 times, using the seed from the configuration. The result is a count of how often each string of bits was produced.
6. **Decode.** Each string of bits is read from right to left, as Qiskit writes it, and the two traces convert every bit back into "*exam* X in slot Y". Saved: `5_solutions.json`, the *schedules* in plain words, along with the rules each one breaks.
7. **Validate.** A classical program tries every possible *schedule* and finds the best ones. The chain then measures how often the circuit found a valid *schedule* and a best *schedule*, and compares this with picking a string of bits at random. Saved: `6_validation.json`.
8. **Map back.** The best *schedule* found is written into the problem model itself, with each *exam* linked to its slot, and checked again against the OCL rules. Saved: `7_problem_solved`.

The whole chain runs with one command, `cd besser && python pipeline.py`, in a few seconds for the five instances.

4 Validation

Missing: How the chains are validated. Waits on the team.
What every run measures: the *Schedule* found, whether it is *feasible* and *optimal* against the *Baseline*, *P(optimal)* and *P(feasible)* over every shot or read, and the odds of a uniformly random bit string, the bar a *Method* must clear. The EMF chain also checks its formulation in every run: a sample's energy is minus the *weight* it places exactly when it breaks no *Constraint*. Each chain is measured on its own instances (Section G); every run is in Section C.

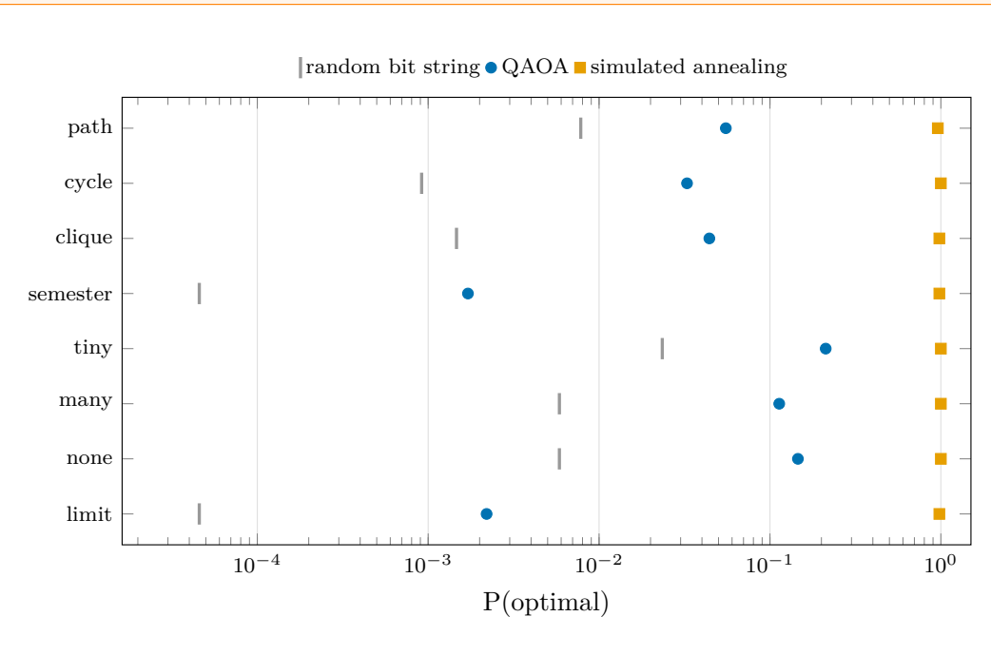


Figure 4: P(optimal) of the EMF chain on each test instance (Section G) by Method, against a uniformly random bit string's, on a log scale; the numbers in Table 5. Solver Configurations in Table 3; from CI run 43071895789.

BESSER

The BESSER chain is validated in two ways: every run is compared against a classical answer that is known to be correct, and a set of automated tests verifies each transformation individually.

The reference answer. A classical program (`besser/tests/baseline.py`) tries every possible *schedule*, assigning one slot to each *exam*, and identifies the valid ones (no *student* with two *exams* in the same slot) and the best ones (the fewest *conflicts*). This is only feasible because the instances are small, but it provides an answer we can trust.

Is the *QUBO* correct? The chain also checks T1 against the reference: the lowest number of points must be w times the fewest *conflicts*, and exactly as many bit strings must reach it as the reference has best *schedules*. To do this, it computes the points of every one of the 2^n bit strings. This holds on all five instances (Table 1, column "QUBO = baseline").

How good is the circuit? Every bit string measured by the simulator is decoded into a *schedule*, and three metrics are computed across all 4096 shots:

- **P(valid)**: how often the circuit produced a valid *schedule*;
- **P(optimal)**: how often it produced one of the best *schedules*;
- **the approximation ratio**: the average penalty of the samples on a scale where 1 is the best possible bit string and 0 the worst, $(E_{\max} - \langle E \rangle)/(E_{\max} - E_{\min})$.

Each metric is compared with picking a bit string at random, which is the baseline the circuit must beat.

Table 1: The BESSER chain on its instances: QAOA with the fixed angles of each *Solver Configuration* (depth 3, 4096 shots, seed 7), against a bit string drawn at random from all 2^n. From `besser/output/summary.md`.

Instance	Qubits	Valid *schedules*	*QUBO* = baseline	P(valid)	P(optimal)	P(optimal), random	Ratio	Ratio, rand
tiny	9	12 of 27	yes	0.3250	0.3250	0.023	0.910	0.7
many	12	24 of 81	yes	0.2012	0.2012	0.0059	0.908	0.7
none	12	0 of 81	yes	0	0.2698	0.0088	0.918	0.7
medium	15	30 of 243	yes	0.0779	0.0779	0.00092	0.901	0.7
limit	18	12 of 729	yes	0.0098	0.0098	4.6e-05	0.911	0.7

On every instance, the circuit finds a best *schedule* far more often than chance (Table 1), but less often as the *problem* grows. The instance `none` has no valid *schedule* at all, so P(valid) is 0 there; its best *schedules* have one *conflict*, and the circuit finds them in a fraction 0.2698 of the shots.

Automated tests. 47 tests (`cd besser && python -m pytest`) check each step on its own: that the penalty points are zero exactly for the valid *schedules* (on `tiny`), that the quantum form of the *QUBO* gives the same value as the *QUBO* on every bit string (on `tiny` and `medium`), that the generated Qiskit code gives each *schedule* the phase its penalty predicts (on `tiny`), that the bits are read in Qiskit's order (right to left), and that two runs of `tiny` produce exactly the same files. To confirm that the tests really catch mistakes, `besser/tests/mutations.py` puts three typical mistakes back, one at a time, into a temporary copy of the chain: the one-hot weight as weak as a *conflict* makes 17 of the 47 tests fail, the flipped sign of the cost layer 4, and count keys read left to right 2 (`besser/README.md`).

5 Limitations

Missing: The limitations. Waits on the team.

- QAOA is far behind *simulated annealing* (Fig. 4), and only simulated, on at most 18 qubits.
- QAOA starts from a fixed ramp of angles on the annealing path [1, §VI], not set by the *Solver Configuration*. From there a deeper circuit raises *P(feasible)* on path and cycle, but there *P(optimal)* stays between 0.028 and 0.062 whatever the depth or optimiser (Tables 3 and 6): the extra *feasible schedules* leave an *exam* out.
- The *simulated annealing* is classical: no quantum annealer was used (issue #23).
- The *Baseline* enumerates every *schedule*, so validation stops at about 10^6 *schedules*.
- The *Schedule* the generated script writes is not validated against its metamodel's OCL.
- The BESSER chain's own.

References

[1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: `1411.4028v1` (cited in §3, §5, §B, §D).

[2] Fred Glover, Gary Kochenberger, and Yu Du. *Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models*. Published in 4OR 17 (2019) 335–371. 2019. arXiv: `1811.11538v6` (cited in §A, §B, §D).

[3] Ali Javadi-Abhari et al. *Quantum computing with Qiskit*. 2024. arXiv: `2405.08810` (cited in §3, §D, §H).

[4] Andrew Lucas. "Ising formulations of many NP problems". In: *Frontiers in Physics* 2 (2014), p. 5. DOI: `10.3389/fphy.2014.00005`. arXiv: `1302.5843v3` (cited in §3, §A, §B).

[5] *`QuantumExamScheduling.ipynb`*. The reference notebook of the paper's formulation, in the repository as `emf/QuantumExamScheduling.ipynb`; cells are numbered by their execution counts (cited in §A, §B, §D).

[6] Mashiyat Zaman, Kotaro Tanahashi, and Shu Tanaka. "PyQUBO: Python Library for Mapping Combinatorial Optimization Problems to QUBO Form". In: *IEEE Transactions on Computers* 71.4 (2022), pp. 838–850. DOI: `10.1109/TC.2021.3063618` (cited in §D, §H).

[7] Ziyu Zhou et al. "Hybrid quantum annealing for large-scale exam scheduling: Validation in real-world educational scenarios". In: *Applied Soft Computing* 184 (2025), p. 113756. DOI: `10.1016/j.asoc.2025.113756` (cited in §1, §2, §3, §A, §B, §D).

A Formulation

A *Problem* lists *students*, the *exams* each of them sits, and a fixed number of *batches*; all *exams* of a *batch* take place at once. Two *exams* that share a *student* are in *Conflict* and must not share a *batch* [7, §5.2, Algorithm 1]. Each *exam* carries a *Weight*, its priority; in the EMF chain, the cost of leaving it unscheduled.

Both chains follow Fig. 1. The *Problem* is formulated as a *QUBO* over one *Assignment Variable* x_{ik} per *exam* i and *batch* k, 1 when *exam* i is in *batch* k [7, §5.3, Eq. (4)], with a penalty per *Constraint*. A *Method* samples the *QUBO*, and a sample is decoded into a *Schedule*: the lowest-energy one in the EMF chain, the most frequent optimal one in BESSER's. Independently of the *QUBO*, a *Baseline* enumerates every *schedule* to find the optimum (the EMF chain's stops at about 10^6, Section 5), and every sample, not only the best, is judged against it (Section 4).

The chains implement two formulations. The paper's [7, §5.3, Eqs. (5)–(10)], which the BESSER chain implements, places each *exam* in exactly one *batch* and has penalties only: a sample of energy zero is a *schedule* without *conflicts*, and nothing ranks one *exam* above another. The EMF chain implements a weighted one (`emf/docs/adr/0009`): each *exam* in at most one *batch* [4, §4.2, Eq. (30)], with every scheduled *exam* earning its *weight*, a weighted set packing [2, §4.2]:

$$H = -\sum_i w_i \sum_k x_{ik} + A\Big(\sum_i \sum_{k<l} x_{ik}x_{il} + \sum_{(i,j)\ \text{in Conflict}} \sum_k x_{ik}x_{jk}\Big).$$

With the *Penalty Coefficient* A above every *weight*, leaving an *exam* unscheduled always costs less than breaking a *Constraint* [4, §4.2], so the optimum is a *feasible Schedule* of least *Cost*, the total *weight* of its *Unscheduled Exams*. The two optima differ wherever not every *exam* fits. On four mutually conflicting *exams* in three *batches* (BESSER's `none`, EMF's `clique`), the paper's formulation, with the penalty BESSER derives for an *exam* left out, above the most *conflicts* placing it can add, accepts one *Conflict*, two *exams* in one *batch*; the weighted one leaves the lightest *exam*, Statistics, unscheduled at *Cost* 1.

The paper samples its *QUBO* on D-Wave's quantum annealer, its Leap Hybrid Sampler and, classically, by *simulated annealing* [7, §5.4], as the notebook does [5, cells 15–17]. The EMF chain samples its own *QUBO* by *simulated annealing* or by QAOA on a simulator; the BESSER chain by QAOA only. *Simulated annealing* needs no D-Wave account, so CI reruns the EMF chain's runs (Table 3), and it gives QAOA a yardstick on the same *QUBO*. Quantum annealing itself is issue #23.

B Rule-to-reference traceability

Table 2: Every ATL rule, template section and runtime step of the EMF chain that implements the formulation or QAOA, with the reference its comment cites, read from the code.

Unit	What it implements, as its comment says
ATL, `problem2conflictgraph.atl`: Problem → Conflict Graph	
`rule Problem2ConflictGraph`	[7, §5.2, Fig. 3]: the conflict graph G, exams as nodes and conflicts as edges.
`rule Problem2ConflictGraph`	[7, §5.3, Eq. (4)]: X has only M columns, so the M batches bound the schedule.
`rule Exam2Exam`	[7, §5.2, Fig. 3]: each exam is a node of G.
`rule Exam2Exam`	[2, §4.2]: the weight w_j of the set packing objective, the exam's priority (ADR-0009).
`helper sharesStudentWith`	[7, §5.2, Algorithm 1, Eq. (3)]: C[i,j] = 1 iff some student sits both exam i and exam j.
`lazy rule Conflict`	[7, §5.2, Fig. 3]: one edge of G per pair with C[i,j] = 1, emitted once, in problem order.
Acceleo, `main.mtl`: Conflict Graph and Solver Configuration → `solve_<name>.py`	
`CONFLICTS`	[7, §5.2, Algorithm 1, Eq. (3)] [5, cell 7]: C, the contradiction matrix, as pairs (i, j), i < j.
`x`	[7, §5.3, Eq. (4)] [5, cell 11]: x[i, k] = 1 iff exam i is in batch k + 1
`constraint1`	[2, §4, known penalties, row 5] [4, §4.2, Eq. (30)]: each exam in at most one batch, the paper's exactly-one [7, §5.3, Eqs. (5)–(7)] relaxed (ADR-0009). With one batch there is no pair to forbid.
`constraint2`	[7, §5.3, Eqs. (8)–(9)] [5, cell 12]: conflicting exams never share a batch, the penalty x[i, k] x[j, k] of [2, §4, known penalties, row 1].
`profit`	[2, §4.2] [4, §4.2, Eq. (31)]: scheduling exam i earns its weight; an unscheduled exam costs it
`A, H, model`	[7, §5.3, Eq. (10)] [5, cell 13]: H, with lambda = gamma = A and the profit added; A is fed from the Solver Configuration
Python, `examschedule.py`: the runtime the generated scripts call	
`schedule_of`	[7, §5.5]: the batch of each exam read off X, x[i, k] = 1 iff exam i is in batch k + 1.
`penalty`	A: the largest weight plus one unless set. An explicit A must exceed every weight, or breaking a constraint can cost less than leaving an exam unscheduled (ADR-0009). [4, §4.2]: "Choosing B < A ensures that it is never favorable to violate the constraint", per weight here. [2, §4, Comment on the Scalar Penalty P]: P large enough that a hard constraint is never broken.
`anneal`	[5, cell 15]: simulated annealing (neal) on the BQM, then pyqubo decodes every read.
`qaoa`	[1, Eqs. (1)–(8)]: QAOA on the Ising form of the compiled model, simulated on Aer, with C = H and the expectation minimised, where Farhi maximises it.
`qaoa`	[4, §2.3, Eq. (12)]: pyqubo's spins, x = (s + 1)/2, so s = 2x - 1.
`qaoa`	[1, Eq. (5)]: |s>, the uniform superposition, top eigenstate of the mixer B
`qaoa`	[1, Eq. (6)]: p alternating cost and mixer layers.
`qaoa`	[1, Eq. (2)]: cost layer U(C, gamma) = exp(-i gamma C), C = H/scale. [4, §1.1, Eq. (4)]: on a qubit Z = 1 - 2x = -s, so a field h on s acts as -h on Z.
`qaoa`	[1, Eqs. (3)–(4)]: mixer layer U(B, beta) = exp(-i beta B), B = sum_j X_j.
`expected_energy`	[1, Eq. (7)]: F_p, the expectation of the cost in the state, estimated from the shots.
`qaoa`	[1, §VI]: a linear ramp (gamma grows, |beta| shrinks) on the Trotterised adiabatic path from |s>, the top of B, to the top of -C, the minimum of C; with the cost layer exp(-i gamma C), that path has gamma and beta of opposite signs.
`qaoa`	[1, Eq. (8)]: the classical loop optimises the 2p angles.

Missing: The BESSER chain's rules and their references. Waits on the BESSER rework.
Once its code cites `[Key locator]` as the EMF chain's does, `generate.py` can read it too.

C Full results

Table 3: Every run of the EMF chain: each test instance solved by both Methods, with the committed seeds; the Penalty Coefficient A used, the Schedule's cost and Unscheduled Exams, the Baseline's optimal cost, and P(optimal) and P(feasible) against a uniformly random bit string's. From CI run 43071895789.

					Optimal	P(optimal)		P(feasible)	
Instance	Var.	Method	A	Result	cost	run	random	run	random
path	8	QAOA, depth 2, COBYLA, 150 iterations, 2048 shots	4	0, none	0	0.055	0.0078	0.752	0.160
path	8	annealing, 50 reads × 10000 sweeps	4	0, none	0	0.960	0.0078	1.000	0.160
cycle	15	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	2	0, none	0	0.033	0.00092	0.749	0.012
cycle	15	annealing, 50 reads × 10000 sweeps	2	0, none	0	1.000	0.00092	1.000	0.012
clique	12	annealing, 50 reads × 10000 sweeps	5	1, Statistics	1	0.980	0.0015	1.000	0.018
clique	12	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	5	1, Statistics	1	0.044	0.0015	0.657	0.018
semester	18	annealing, 50 reads × 10000 sweeps	4	0, none	0	0.980	0.000046	1.000	0.0034
semester	18	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	4	0, none	0	0.0017	0.000046	0.611	0.0034
tiny	9	annealing, 50 reads × 10000 sweeps	2	0, none	0	1.000	0.023	1.000	0.084
tiny	9	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	2	0, none	0	0.212	0.023	0.807	0.084
many	12	annealing, 50 reads × 10000 sweeps	2	0, none	0	1.000	0.0059	1.000	0.035
many	12	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	2	0, none	0	0.113	0.0059	0.681	0.035
none	12	annealing, 50 reads × 10000 sweeps	2	1, Algebra	1	1.000	0.0059	1.000	0.018
none	12	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	2	1, Statistics	1	0.146	0.0059	0.771	0.018
limit	18	annealing, 50 reads × 10000 sweeps	2	0, none	0	0.980	0.000046	1.000	0.0034
limit	18	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	2	0, none	0	0.0022	0.000046	0.650	0.0034

Table 4: The Schedule each run of Table 3 found: the exams of each batch.

Instance	Method	Batch: exams	Unscheduled
path	QAOA	1: Calculus, Chemistry; 2: Algebra, Physics	none
path	annealing	1: Calculus, Chemistry; 2: Algebra, Physics	none
cycle	QAOA	1: Calculus; 2: Algebra, Chemistry; 3: Physics, Programming	none
cycle	annealing	1: Chemistry; 2: Calculus, Programming; 3: Algebra, Physics	none
clique	annealing	1: Physics; 2: Calculus; 3: Algebra	Statistics
clique	QAOA	1: Physics; 2: Algebra; 3: Calculus	Statistics
semester	annealing	1: Physics, Programming; 2: Calculus, Databases; 3: Algebra, Chemistry	none
semester	QAOA	1: Calculus, Databases; 2: Algebra, Chemistry; 3: Physics, Programming	none
tiny	annealing	2: Physics; 3: Math, Chemistry	none
tiny	QAOA	1: Math, Chemistry; 2: Physics	none
many	annealing	1: Chemistry; 2: Calculus; 3: Algebra, Physics	none
many	QAOA	1: Calculus, Chemistry; 3: Algebra, Physics	none
none	annealing	1: Physics; 2: Statistics; 3: Calculus	Algebra
none	QAOA	1: Physics; 2: Calculus; 3: Algebra	Statistics
limit	annealing	1: Physics, Programming; 2: Calculus, Databases; 3: Algebra, Chemistry	none
limit	QAOA	1: Algebra, Programming; 2: Calculus, Databases; 3: Physics, Chemistry	none

Table 5: P(optimal) of the EMF chain on each test instance (Section G) by Method, against a uniformly random bit string's; in brackets, how many times chance. Solver Configurations in Table 3; from CI run 43071895789.

Instance	Variables	Random	QAOA	Simulated annealing
path	8	0.0078	0.055 (7×)	0.960 (123×)
cycle	15	0.00092	0.033 (36×)	1.000 (1092×)
clique	12	0.0015	0.044 (30×)	0.980 (669×)
semester	18	0.000046	0.0017 (37×)	0.980 (21408×)
tiny	9	0.023	0.212 (9×)	1.000 (43×)
many	12	0.0059	0.113 (19×)	1.000 (171×)
none	12	0.0059	0.146 (25×)	1.000 (171×)
limit	18	0.000046	0.0022 (48×)	0.980 (21408×)

Table 6: The EMF chain with other QAOA settings, every one reaching the *optimal cost*. Measured locally on macOS (arm64) with the pinned `emf/requirements.txt`, not in CI, so a rerun on Linux may differ in the second digit; copied by hand from `emf/CONTEXT.md`. COBYLA stops well before 150 iterations (29 to 54 evaluations); a Powell iteration is a line search over every angle (398 evaluations on cycle).

Instance	Variables	*Method*	*P(optimal)*	*P(feasible)*
path	8	QAOA, depth 1, COBYLA, 150 iterations, 2048 shots	0.031	0.723
path	8	QAOA, depth 3, COBYLA, 150 iterations, 2048 shots	0.062	0.881
path	8	QAOA, depth 2, Nelder-Mead, 150 iterations, 2048 shots	0.049	0.796
cycle	15	QAOA, depth 1, COBYLA, 150 iterations, 4096 shots	0.028	0.517
cycle	15	QAOA, depth 3, COBYLA, 150 iterations, 4096 shots	0.047	0.865
cycle	15	QAOA, depth 2, Powell, 150 iterations, 4096 shots	0.036	0.809

BESSER chain

Every instance was run with the same *Solver Configuration* (`besser/config/<instance>.py`): *conflict* weight $w = 1$, depth 3, 4096 shots and seed 7, using the angles of a linear ramp, identical for every instance: $\gamma = (0.2667, 0.5333, 0.8)$ and $\beta = (0.8, 0.5333, 0.2667)$. The weight of the one-slot rule, λ, is not configured: T1 computes it as $(\Delta + 1)\,w$, where Δ is the largest number of *conflicts* of a single *exam*.

Section C shows what each run produced: the size of the circuit, how many different bit strings appeared in the 4096 shots, and the *schedule* the chain wrote back into the problem model, which is the best *schedule* that was measured most often.

Instance	Qubits	Δ	λ	Operations	Bit strings	*Schedule* found	Shots	*Conflicts*
tiny	9	2	3	207	269	Mon_0900: Chemistry; Mon_1400: Physics; Tue_0900: Math	130	0
many	12	2	3	285	846	Mon_0900: Physics; Mon_1400: Algebra, Chemistry; Tue_0900: Calculus	44	0
none	12	3	4	366	855	Mon_0900: Calculus, Physics; Mon_1400: Statistics; Tue_0900: Algebra	46	1
medium	15	2	3	390	1787	Mon_0900: Algebra, Physics; Mon_1400: Chemistry; Tue_0900: Calculus, Programming	17	0
limit	18	3	4	522	2826	Mon_0900: Algebra, Chemistry; Mon_1400: Physics, Programming; Tue_0900: Calculus, Databases	6	0

On every instance, the chain found and wrote back a best *schedule*: a valid one on the four instances that have one, and on `none`, which has none, a *schedule* with a single *conflict* (Calculus and Physics in the same slot), which is the best possible there. As the *problem* grows, the 4096 shots spread across more and more bit strings (from 269 on `tiny` to 2826 on `limit`), so the *schedule* found appears fewer times: 130 shots on `tiny`, only 6 on `limit`. The chain still finds it, but with fewer shots it could be missed; this reflects the trend of P(optimal) in Table 1.

D EMF chain in detail

This appendix expands Section 2: how the EMF chain turns one `.exam` file into a Python script in three model-driven steps, run headless by an Eclipse application; the script then solves the *Problem* and validates what it finds. The models parsed from the file, and the *Conflict Graph*, are checked against their metamodels' multiplicities and OCL invariants (Section E) before the next step uses them.

Xtext. The grammar imports the hand-written *Problem* and *Solver Configuration* metamodels, so one `.exam` file holds a *Problem* (its *batches*, its *students*, and each *exam* with its optional *weight*, 1 by default, and the *students* who sit it) and, optionally, its *Solver Configuration*: the *Method*, the *Method*'s settings, the seed and the *Penalty Coefficient*, automatic unless given (Section G).

ATL. An EMFTVM transformation makes the *Problem* a *Conflict Graph* [7, §5.2, Fig. 3]. It copies the *exams* with their *weights* and the number of *batches* M, the range of the index k of x_{ik} [7, §5.3, Eq. (4)], and adds one *Conflict* for each pair of *exams* that share a *student*, the paper's course conflict matrix C [7, §5.2, Algorithm 1, Eq. (3); 5, cell 7].

Acceleo. A template generates `solve_<name>.py` from the *Conflict Graph* and the *Solver Configuration*: both as Python data, then the notebook's pyqubo construction [5, cells 11–13; 6] (`emf/docs/adr/0007`), made the weighted set packing of Section A (`emf/docs/adr/0009`). Each *Constraint* is labelled in domain terms, `at-most-one-batch-<exam>` or `conflict-<exam>-<exam>-batch-<k>`, so a broken one names its *exams*. pyqubo compiles H into the *QUBO* and its Ising form, so the chain models neither.

The runtime. The generated script holds only what depends on the models; it imports the hand-written `emf/python/examschedule.py`, which holds the rest (`emf/docs/adr/0005`). It sets A to the largest *weight* plus one, or rejects an explicit A not above every *weight* [2, §4, Comment on the Scalar Penalty P], and samples by the *Method* chosen. *Simulated annealing* runs dwave-neal on the compiled model, as the notebook does [5, cell 15]. QAOA [1] builds the *QAOA Circuit* from the Ising form, with p alternating cost and mixer layers on the uniform superposition [1, Eqs. (2)–(6)], runs it with Qiskit [3] on the Aer simulator, and lets a classical optimiser tune the $2p$ angles to minimise the expected energy (Farhi et al. maximise theirs) [1, Eqs. (7)–(8)], starting on the annealing path [1, §VI]. The runtime then decodes the lowest-energy sample into a *Schedule* model [7, §5.5], runs the *Baseline*, and writes the run's results, from which Table 3 is generated.

Every ATL rule, and every template section and runtime step that implements the formulation or QAOA, cites its reference in a comment beside it, as `[Key locator]`; Table 2 is generated from those comments.

E Metamodels

Containment is drawn with a diamond at the container, a reference with an open arrow; *inv* names an OCL invariant.

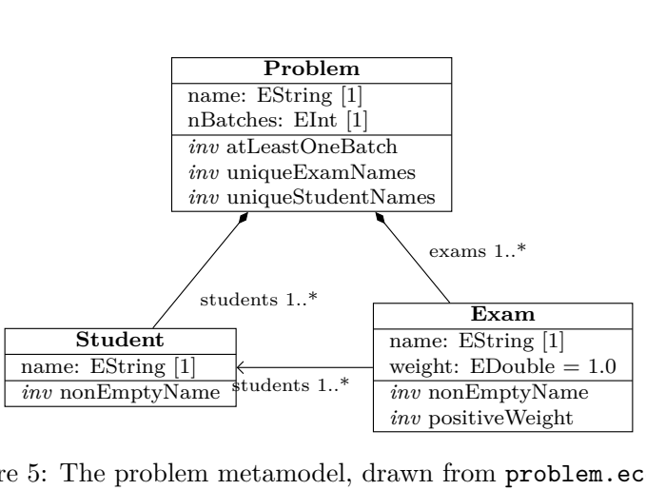


Figure 5: The problem metamodel, drawn from `problem.ecore`.

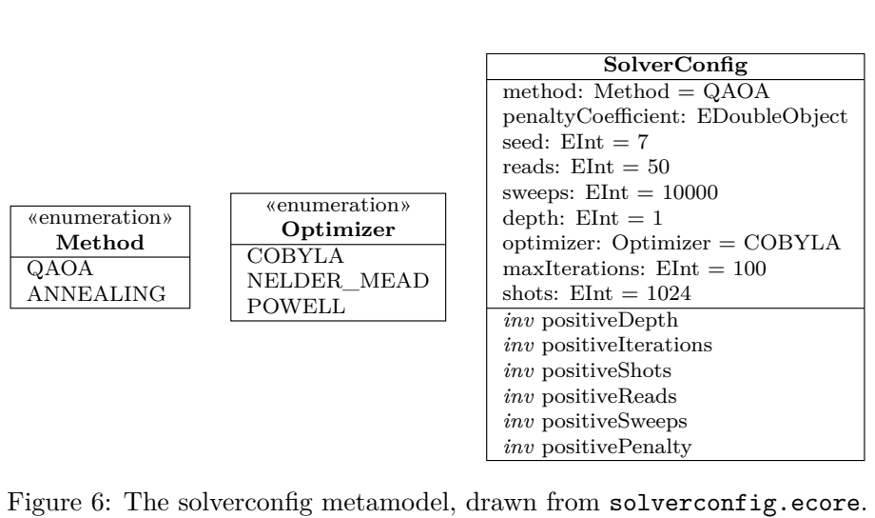


Figure 6: The solverconfig metamodel, drawn from `solverconfig.ecore`.

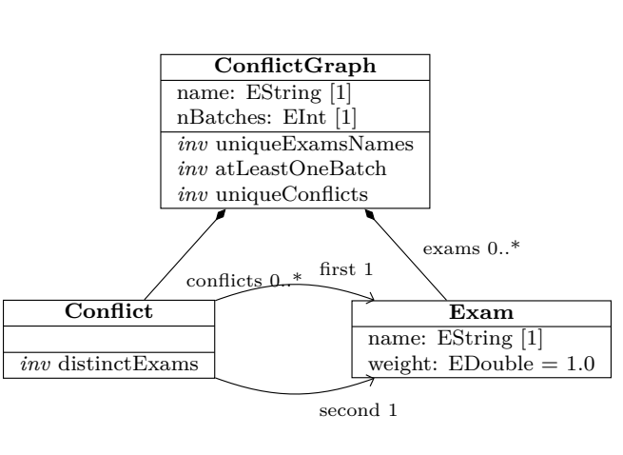


Figure 7: The conflictgraph metamodel, drawn from `conflictgraph.ecore`.

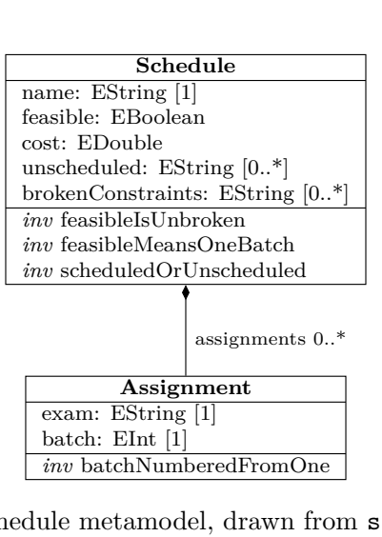


Figure 8: The schedule metamodel, drawn from `schedule.ecore`.

Missing: The BESSER chain's metamodels. Waits on the BESSER rework.
Part I's diagrams (`docs/imgs/problem-dsl-01.png`, `docs/imgs/solution-dsl-01.png`) once BESSER settles, with its *Solver Configuration* and quantum metamodels.

F Glossary

Problem

Problem One instance to be scheduled: a set of *students*, a set of *exams*, and the number of *batches* available.

Student A person who must sit zero or more *exams*; identified by a unique name.

Exam A single final exam sat by one or more *students*; identified by a unique name and carrying a *weight*.

Weight The priority of an *exam*: the cost of leaving it unscheduled. Always positive; 1 when not given, so by default a *schedule* places as many *exams* as it can.

Avoid: profit, importance

Batch One of a fixed number of exam periods; all *exams* in the same batch take place simultaneously. The "slot" in the problem's name, Exam-Slot Assignment, is a batch.

Avoid: slot, time slot, TimeSlot, period

Conflict Graph

Conflict A relation between two distinct *exams* that share at least one *student*, so they must not be in the same *batch*.

Avoid: contradiction, clash

Conflict Graph The *exams* as nodes and their *conflicts* as edges, together with the number of *batches*.

Avoid: course conflict, contradiction matrix

Solving

Solver Configuration The choices about how a *problem* is solved rather than what the *problem* is: the *method*, its settings, the seed and the *penalty coefficient*.

Avoid: settings, parameters

Method How the compiled formulation is sampled: QAOA or *simulated annealing*.

Avoid: solver (ambiguous with the Solver Configuration*), backend*

Assignment Variable The binary decision x[Exam, Batch]: 1 when that *exam* is placed in that *batch*. pyqubo labels it `c[i][k]`, as in the notebook.

Avoid: qubit (that is its position in the QAOA circuit*)*

Hamiltonian The energy to minimise, H = `profit` + A · (`constraint1` + `constraint2`), with `profit` = −Σ *weights* of scheduled *exams*; `constraint1` keeps each *exam* in at most one *batch*, `constraint2` keeps conflicting *exams* out of the same *batch*.

Avoid: cost function, objective (for the whole)

Constraint One pyqubo `Constraint` term of the *Hamiltonian*, labelled in domain terms: `at-most-one-batch-<exam>`, or `conflict-<exam>-<exam>-batch-<k>`.

Avoid: penalty term

Penalty Coefficient The value fed to the *Hamiltonian*'s placeholder A. Automatic by default, the largest *weight* plus one; an explicit value must exceed every *weight*.

Avoid: lambda, penalty weight

QUBO The *Hamiltonian* as pyqubo compiles it: a quadratic polynomial over the *assignment variables*. It exists only at run time, inside the generated script.

Avoid: QUBO matrix (BQM is pyqubo's name for the same thing)

QAOA Circuit The quantum circuit built from the Ising form of the compiled model: initial superposition, p alternating cost and mixer layers, and measurement.

Avoid: ansatz, program, quantum model

Simulated Annealing The classical sampler the notebook uses (`neal`): a number of reads, each a run of a number of sweeps.

Avoid: quantum annealing (the paper's D-Wave hardware, which the chain does not use)

Solution

Schedule The outcome of solving a *problem*: the *batch* of each scheduled *exam*, as decoded from the lowest-energy sample.

Avoid: plan, solution matrix

Unscheduled Exam An *exam* the *schedule* places in no *batch*, accepted at the cost of its *weight*.

Avoid: missed exam, dropped exam

Cost (of a schedule) The total *weight* of its *unscheduled exams*.

Feasible (schedule) Breaking no *constraint*: each *exam* in at most one *batch* and no *conflict* within a *batch*. *Unscheduled exams* are allowed. An infeasible *schedule* lists its broken *constraints*.

Avoid: valid (the BESSER chain's word for a schedule that places every exam with no conflict: here, feasible *with no* unscheduled exam*)*

Optimal (schedule) *Feasible*, with the lowest *cost* any *feasible schedule* of the *problem* has.

Validation

Baseline Enumerating every *schedule* (each *exam* in one *batch* or none), independently of the *QUBO*, to find the *optimal cost*.

Avoid: reference solution, ground truth

P(optimal) The fraction of QAOA shots or annealing reads that decode to an *optimal schedule*.

Avoid: success rate

P(feasible) The fraction of QAOA shots or annealing reads that decode to a *feasible schedule*.

Avoid: success rate

G Test instances

The EMF chain's instances, `emf/instances/`. Each has a twin, `<n>-<name>-qaoa.exam` or `<n>-<name>-annealing.exam`, holding the same *problem* solved by the other *Method*.

1-path.exam

```
// Q16 (i): four exams whose conflicts form a path, two batches. Feasible (a path is 2-colourable).
problem path batches 2
students Ana Rui Eva
exam Algebra   weight 2 { Ana }
exam Calculus  weight 3 { Ana Rui }
exam Physics   weight 1 { Rui Eva }
exam Chemistry weight 2 { Eva }

solver { method QAOA depth 2 optimizer COBYLA iterations 150 shots 2048 seed 7 }
```

2-cycle.exam

```
// BESSER's medium (besser/instances/medium.py) is this problem.
// Q16 (ii): five exams whose conflicts form a 5-cycle, three batches. Feasible (needs 3 batches).
// No weights: every exam weighs 1, so the schedule places as many exams as it can.
problem cycle batches 3
students Ana Rui Eva Ivo Lia
exam Algebra { Ana Lia }
exam Calculus { Ana Rui }
exam Physics { Rui Eva }
exam Chemistry { Eva Ivo }
exam Programming { Ivo Lia }

solver { method QAOA depth 2 optimizer COBYLA iterations 150 shots 4096 seed 7 }
```

3-clique.exam

```
// Q16 (iii): four exams that all conflict, three batches. No schedule places them all: the
// lowest-weight exam, Statistics, is left unscheduled, at cost 1 (ADR-0009).
problem clique batches 3
students Ana Rui Eva
exam Algebra    weight 4 { Ana Rui Eva }
exam Calculus   weight 3 { Ana Rui Eva }
exam Physics    weight 2 { Ana Rui Eva }
exam Statistics weight 1 { Ana Rui Eva }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

4-semester.exam

```
// Q16 (iv): six exams of a semester, three batches. Feasible.
problem semester batches 3
students Ana Rui Eva Ivo Lia Tom
exam Algebra     weight 3 { Ana Rui }
exam Calculus    weight 3 { Rui Eva Tom }
exam Physics     weight 2 { Ana Eva }
exam Chemistry   weight 2 { Ivo Lia }
exam Programming weight 1 { Lia Tom }
exam Databases   weight 1 { Ivo Ana }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

Instances 5 to 8 are the BESSER chain's instances, `besser/instances/`, written in the Exam DSL, and `2-cycle` is its `medium`.

5-tiny.exam

```
// BESSER's tiny (besser/instances/tiny.py), Part I's hand-made example, in the Exam DSL.
// Three exams whose conflicts form a path, three batches, no weights. Feasible: cost 0.
problem tiny batches 3
students Alice Bob Carla David Eva
exam Math      { Alice Carla }
exam Physics   { Alice Bob Eva }
exam Chemistry { Bob David }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

6-many.exam

```
// BESSER's many (besser/instances/many.py) in the Exam DSL: 1-path in three batches, no weights.
// Four exams whose conflicts form a path. Feasible: cost 0.
problem many batches 3
students Ana Rui Eva
exam Algebra   { Ana }
exam Calculus  { Ana Rui }
exam Physics   { Rui Eva }
exam Chemistry { Eva }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

7-none.exam

```
// BESSER's none (besser/instances/none.py) in the Exam DSL: 3-clique with no weights.
// Four exams that all conflict, three batches. No schedule places them all: one exam is left
// unscheduled, at cost 1 (ADR-0009), where BESSER's formulation accepts one conflict instead.
problem none batches 3
students Ana Rui Eva
exam Algebra    { Ana Rui Eva }
exam Calculus   { Ana Rui Eva }
exam Physics    { Ana Rui Eva }
exam Statistics { Ana Rui Eva }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

8-limit.exam

```
// BESSER's limit (besser/instances/limit.py) in the Exam DSL: 4-semester with no weights.
// Six exams of a semester, three batches. Feasible: cost 0.
problem limit batches 3
students Ana Rui Eva Ivo Lia Tom
exam Algebra     { Ana Rui }
exam Calculus    { Rui Eva Tom }
exam Physics     { Ana Eva }
exam Chemistry   { Ivo Lia }
exam Programming { Lia Tom }
exam Databases   { Ivo Ana }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

H Reproducing the results

- The EMF chain: `cd emf && ./build.sh` (JDK 21+, Python 3.12) builds the chain and runs it on every instance, writing `emf/out/<name>/results_<name>.json`. The Python packages, pyqubo [6], Qiskit [3], Qiskit Aer and dwave-neal among them, are pinned in `emf/requirements.txt`. CI runs the same on every pull request and push to main that changes `emf/` (`.github/workflows/emf.yml`) and uploads the results as the `emf-results` artifact.
- This report: `docs/report_2/build.sh` (TeX Live, `gh`) fetches those results from CI, generates its tables, Section B and the diagrams of Section E, and runs latexmk; it does not run the chains.
- The LLM logs of Part II: `docs/llm_usage_2.md`, with the session records it links in `docs/assets/`, and `besser/docs/llm_usage_2.md`.

Missing: How to reproduce the BESSER chain's results. Waits on the BESSER rework.
`besser/README.md`'s commands, once the chain settles.